

MultiAncestryPGxGWAS: Toy Data Quick Start

Wujuan Zhong, Judong Shen

2026-10-05

Overview

This vignette uses bundled toy files under `inst/extdata` and runs the full method set demonstrated in the simulation-application vignette for both continuous and binary outcomes.

- Continuous methods: LR-Sandwich (all samples and per ancestry), fixed-effects meta-analysis, MR-MEGA, fastGWA_GE, GENESIS, and LEMMA.
- Binary methods: fastGWA_GE (naive 0/1), LEMMA (naive 0/1), GENESIS, GEM, and MR-MEGA of per-ancestry GEM outputs.

We first set up paths to external tools and runtime directories.

```
library(MultiAncestryPGxGWAS)

configs <- list(
  plink1.9.path = "plink",
  plink2.path = "plink2",
  flashpca.path = "flashpca_x86-64",
  gcta.path = "gcta",
  lemma.path = "lemma_1_0_4",
  gem.path = "GEM_1.4.5_static",
  bgenix.path = "bgenix",
  mrmega.path = "MR-MEGA",
  mpirun.path = "mpirun"
)
mpgx_validate_tool_configs(configs)
#> OK

work_dir <- file.path(tempdir(), "MultiAncestryPGxGWAS_toy_methods")
dir.create(work_dir, recursive = TRUE, showWarnings = FALSE)

method_out_dirs <- list(
  continuous = list(
    root = file.path(work_dir, "continuous"),
    lr_sandwich = file.path(work_dir, "continuous", "lr_sandwich"),
    fe_meta_lr = file.path(work_dir, "continuous", "fe_meta_lr"),
    mrmega_lr = file.path(work_dir, "continuous", "mrmega_lr"),
    fastgwa_ge = file.path(work_dir, "continuous", "fastgwa_ge"),
    genesis = file.path(work_dir, "continuous", "genesis"),
    lemma = file.path(work_dir, "continuous", "lemma")
  ),
  binary = list(
    root = file.path(work_dir, "binary"),
```

```

fastgwa_ge_naive = file.path(work_dir, "binary", "fastgwa_ge_naive"),
lemma_naive = file.path(work_dir, "binary", "lemma_naive"),
genesis = file.path(work_dir, "binary", "genesis"),
gem = file.path(work_dir, "binary", "gem"),
mrmega_gem = file.path(work_dir, "binary", "mrmega_gem")
)
)

method_leaf_dirs <- unique(c(
  unlist(method_out_dirs$continuous, use.names = FALSE),
  unlist(method_out_dirs$binary, use.names = FALSE)
))
invisible(lapply(method_leaf_dirs, dir.create, recursive = TRUE, showWarnings = FALSE))

```

Then we locate bundled toy inputs and prepare ancestry-specific keep files.

```

example_path <- function(rel) mpgx_example_file(rel)

toy_paths <- list(
  bfile_prefix = sub("\\.bed$", "", example_path("genotype_data/toyData.bed")),
  ancestry_info_file = example_path("genotype_data/toyData.ancestry.txt"),
  sparse_grm_prefix =
    sub("\\.grm\\.sp$", "", example_path("genotype_artifacts/toyData.sp_grm.grm.sp")),
  bgen_file = example_path("genotype_artifacts/toyData.bgen"),
  gds_file = example_path("genotype_artifacts/toyData.gds"),
  cont = list(
    phenotype_file =
      example_path("simulated_data/continuous/sim_phenotype_seed_100001.txt"),
    qcovar_file = example_path("simulated_data/continuous/sim_qcovar.txt"),
    covar_file = example_path("simulated_data/continuous/sim_covar.txt"),
    envir_file = example_path("simulated_data/continuous/sim_envir.txt"),
    lemma_phenotype_file =
      example_path("simulated_data/continuous/sim_phenotype_forLEMMA_seed_100001.txt"),
    lemma_envir_file = example_path("simulated_data/continuous/sim_envir_forLEMMA.txt"),
    lemma_covar_file =
      example_path("simulated_data/continuous/sim_covar_qcovar_forLEMMA.txt")
  ),
  bin = list(
    phenotype_file = example_path("simulated_data/binary/sim_phenotype_seed_100001.txt"),
    qcovar_file = example_path("simulated_data/binary/sim_qcovar.txt"),
    covar_file = example_path("simulated_data/binary/sim_covar.txt"),
    envir_file = example_path("simulated_data/binary/sim_envir.txt"),
    lemma_phenotype_file =
      example_path("simulated_data/binary/sim_phenotype_forLEMMA_seed_100001.txt"),
    lemma_envir_file = example_path("simulated_data/binary/sim_envir_forLEMMA.txt"),
    lemma_covar_file = example_path("simulated_data/binary/sim_covar_qcovar_forLEMMA.txt"),
    gem_phenotype_file =
      example_path("simulated_data/binary/sim_phenotype_forGEM_seed_100001.txt")
  )
)

cont_keep <- mpgx_prepare_ancestry_keep_files(
  ancestry_info_file = toy_paths$ancestry_info_file,

```

```

    out_dir = method_out_dirs$continuous$root
  )

bin_keep <- mpgx_prepare_ancestry_keep_files(
  ancestry_info_file = toy_paths$ancestry_info_file,
  out_dir = method_out_dirs$binary$root
)

cont_inputs <- list(
  bfile_prefix = toy_paths$bfile_prefix,
  sparse_grm_prefix = toy_paths$sparse_grm_prefix,
  bgen_file = toy_paths$bgen_file,
  gds_file = toy_paths$gds_file,
  phenotype_file = toy_paths$cont$phenotype_file,
  qcovar_file = toy_paths$cont$qcovar_file,
  covar_file = toy_paths$cont$covar_file,
  envir_file = toy_paths$cont$envir_file,
  lemma_pheno = toy_paths$cont$lemma_phenotype_file,
  lemma_envir = toy_paths$cont$lemma_envir_file,
  lemma_covar = toy_paths$cont$lemma_covar_file,
  keep_files = cont_keep$keep_files
)

bin_inputs <- list(
  bfile_prefix = toy_paths$bfile_prefix,
  sparse_grm_prefix = toy_paths$sparse_grm_prefix,
  bgen_file = toy_paths$bgen_file,
  gds_file = toy_paths$gds_file,
  phenotype_file = toy_paths$bin$phenotype_file,
  qcovar_file = toy_paths$bin$qcovar_file,
  covar_file = toy_paths$bin$covar_file,
  envir_file = toy_paths$bin$envir_file,
  lemma_pheno = toy_paths$bin$lemma_phenotype_file,
  lemma_envir = toy_paths$bin$lemma_envir_file,
  lemma_covar = toy_paths$bin$lemma_covar_file,
  gem_phenotype_file = toy_paths$bin$gem_phenotype_file,
  keep_files = bin_keep$keep_files
)

```

Continuous outcome methods

We first run LR-Sandwich on all samples and then per ancestry to create inputs for meta-analysis methods.

```

lr_sandwich_all_prefix <- file.path(
  method_out_dirs$continuous$lr_sandwich,
  "continuous.LRallSandwich"
)

lr_sandwich_all <- mpgx_run_lr_sandwich(
  bfile_prefix = cont_inputs$bfile_prefix,
  phenotype_file = cont_inputs$phenotype_file,
  qcovar_file = cont_inputs$qcovar_file,

```

```

    covar_file = cont_inputs$covar_file,
    envir_file = cont_inputs$envir_file,
    output_prefix = lr_sandwich_all_prefix,
    sandwich = TRUE,
    configs = configs
  )

ancestry_levels <- names(cont_inputs$keep_files)
lr_sandwich_by_eth <- lapply(ancestry_levels, function(ances) {
  out_prefix <- file.path(
    method_out_dirs$continuous$lr_sandwich,
    paste0("continuous.LRallSandwich.", ances)
  )

  mpgx_run_lr_sandwich(
    bfile_prefix = cont_inputs$bfile_prefix,
    phenotype_file = cont_inputs$phenotype_file,
    qcovar_file = cont_inputs$qcovar_file,
    covar_file = cont_inputs$covar_file,
    envir_file = cont_inputs$envir_file,
    keep_file = cont_inputs$keep_files[[ances]],
    output_prefix = out_prefix,
    sandwich = TRUE,
    configs = configs
  )
})
names(lr_sandwich_by_eth) <- ancestry_levels
lr_sandwich_by_eth_files <- extract_gwas_files(lr_sandwich_by_eth)

```

We then combine per-ancestry LR-Sandwich outputs using fixed-effects meta-analysis and MR-MEGA.

```

meta_lr_cont_file <- file.path(
  method_out_dirs$continuous$fe_meta_lr,
  "continuous.meta_LRallSandwich.txt"
)

meta_lr_cont <- mpgx_run_fe_meta_for_lr(
  ancestry_result_files = lr_sandwich_by_eth_files,
  ancestry_labels = ancestry_levels,
  output_file = meta_lr_cont_file
)

mrmega_lr_cont <- mpgx_run_mrmega_for_lr(
  ancestry_result_files = lr_sandwich_by_eth_files,
  output_prefix_base = file.path(
    method_out_dirs$continuous$mrmega_lr,
    "continuous.MRMEGA_LRallSandwich"
  ),
  configs = configs
)

```

Next, run fastGWA_GE for the continuous phenotype using the precomputed sparse GRM.

```
fastgwa_cont <- mpgx_run_fastgwa_ge(  
  bfile_prefix = cont_inputs$bfile_prefix,  
  grm_sparse_prefix = cont_inputs$sparse_grm_prefix,  
  phenotype_file = cont_inputs$phenotype_file,  
  qcovar_file = cont_inputs$qcovar_file,  
  covar_file = cont_inputs$covar_file,  
  envir_file = cont_inputs$envir_file,  
  output_prefix = file.path(method_out_dirs$continuous$fastgwa_ge, "continuous.fastGWA_GE"),  
  sandwich = TRUE,  
  configs = configs  
)
```

Then run GENESIS on the same continuous-outcome inputs for comparison.

```
genesis_cont <- mpgx_run_genesis(  
  gds_file = cont_inputs$gds_file,  
  sparse_grm_prefix = cont_inputs$sparse_grm_prefix,  
  phenotype_file = cont_inputs$phenotype_file,  
  qcovar_file = cont_inputs$qcovar_file,  
  covar_file = cont_inputs$covar_file,  
  envir_file = cont_inputs$envir_file,  
  output_file = file.path(method_out_dirs$continuous$genesis, "continuous.GENESIS.txt")  
)
```

Finally, run LEMMA for continuous outcome analysis with the LEMMA-formatted inputs.

```
lemma_cont <- mpgx_run_lemma(  
  bgen_file = cont_inputs$bgen_file,  
  phenotype_file = cont_inputs$lemma_pheno,  
  envir_file = cont_inputs$lemma_envir,  
  covar_file = cont_inputs$lemma_covar,  
  output_prefix = file.path(method_out_dirs$continuous$lemma, "continuous.LEMMA"),  
  n_tasks = 1,  
  configs = configs  
)
```

Binary outcome methods

We run binary-outcome methods, including the naive 0/1 treatment for fastGWA_GE and LEMMA. Note: fastGWA_GE and LEMMA naively treat the binary outcome as 0/1.

First, run fastGWA_GE naively with the binary phenotype coded as 0/1.

```
fastgwa_bin_naive <- mpgx_run_fastgwa_ge(  
  bfile_prefix = bin_inputs$bfile_prefix,  
  grm_sparse_prefix = bin_inputs$sparse_grm_prefix,  
  phenotype_file = bin_inputs$phenotype_file,
```

```

qcovar_file = bin_inputs$qcovar_file,
covar_file = bin_inputs$covar_file,
envir_file = bin_inputs$envir_file,
output_prefix =
  file.path(method_out_dirs$binary$fastgwa_ge_naive, "binary.fastGWA_GE_naive"),
sandwich = TRUE,
configs = configs
)

```

Next, run LEMMA naively with the binary phenotype coded as 0/1.

```

lemma_bin_naive <- mpgx_run_lemma(
  bgen_file = bin_inputs$bgen_file,
  phenotype_file = bin_inputs$lemma_pheno,
  envir_file = bin_inputs$lemma_envir,
  covar_file = bin_inputs$lemma_covar,
  output_prefix = file.path(method_out_dirs$binary$lemma_naive, "binary.LEMMA_naive"),
  n_tasks = 1,
  configs = configs
)

```

Then run GENESIS for binary-outcome analysis.

```

genesis_bin <- mpgx_run_genesis(
  gds_file = bin_inputs$gds_file,
  sparse_grm_prefix = bin_inputs$sparse_grm_prefix,
  phenotype_file = bin_inputs$phenotype_file,
  qcovar_file = bin_inputs$qcovar_file,
  covar_file = bin_inputs$covar_file,
  envir_file = bin_inputs$envir_file,
  output_file = file.path(method_out_dirs$binary$genesis, "binary.GENESIS.txt")
)

```

Next, run GEM for binary-outcome analysis.

```

gem_bin <- mpgx_run_gem(
  bfile_prefix = bin_inputs$bfile_prefix,
  phenotype_file = bin_inputs$gem_phenotype_file,
  output_prefix = file.path(method_out_dirs$binary$gem, "binary.GEM"),
  configs = configs
)

```

Finally, prepare per-ancestry GEM inputs and run MR-MEGA across ancestries.

```

gem_by_eth_files <- mpgx_prepare_mrmega_gem_inputs(
  bfile_prefix = bin_inputs$bfile_prefix,
  gem_phenotype_file = bin_inputs$gem_phenotype_file,
  keep_files = bin_inputs$keep_files,
  output_dir = method_out_dirs$binary$mrmega_gem,
  configs = configs
)

```

```
pc_num_bin <- max(length(gem_by_eth_files) - 3, 0)

mrmega_gem <- mpgx_run_mrmega_for_gem(
  ancestry_result_files = gem_by_eth_files,
  output_prefix_base = file.path(method_out_dirs$binary$mrmega_gem, "binary.MRMEGA_GEM"),
  configs = configs
)
```

Summarize method outputs

This final section summarizes parsed GWAS outputs across all continuous and binary methods.

```
read_causal_snps <- function(trait_dir) {
  snp_files <- list.files(
    trait_dir,
    pattern = "^SNPs_(maineffects|interaction)\\.\"",
    full.names = TRUE
  )
  unique(unlist(lapply(snp_files, function(f) {
    dat <- utils::read.table(f, stringsAsFactors = FALSE)
    as.character(dat[[1]])
  })), use.names = FALSE))
}

summarize_method_list <- function(
  method_results,
  causal_snps,
  scenario,
  seed,
  mrmega_methods = character(),
  mrmega_pc_num = NULL
) {
  summaries <- lapply(names(method_results), function(method_name) {
    method_df <- mpgx_get_method_gwas_df(method_results[[method_name]], label = method_name)
    mrmega_pc <- if (method_name %in% mrmega_methods) mrmega_pc_num else NULL
    has_non_missing_2df <- "P_2df" %in% colnames(method_df) && any(!is.na(method_df$P_2df))
    summary_df <- mpgx_summarize_association(
      result = method_df,
      causal_snps = causal_snps,
      rm_na_2df = has_non_missing_2df,
      mrmega_pc_num = mrmega_pc
    )

    cbind(
      data.frame(
        method = method_name,
        scenario = scenario,
        seed = seed,
        stringsAsFactors = FALSE
      ),
      summary_df
    )
  })
}
```

```

    )
  })

  out <- do.call(rbind, summaries)
  rownames(out) <- NULL
  out
}

causal_cont <- read_causal_snps(dirname(toy_paths$cont$phenotype_file))
causal_bin <- read_causal_snps(dirname(toy_paths$bin$phenotype_file))

continuous_methods <- list(
  LRallSandwich = lr_sandwich_all,
  meta_LRallSandwich = meta_lr_cont,
  MRMEGA_LRallSandwich = mrmega_lr_cont,
  fastGWA_GE = fastgwa_cont,
  GENESIS = genesis_cont,
  LEMMA = lemma_cont
)

continuous_summary <- summarize_method_list(
  method_results = continuous_methods,
  causal_snps = causal_cont,
  scenario = "toy_continuous",
  seed = 100001,
  mrmega_methods = "MRMEGA_LRallSandwich",
  mrmega_pc_num = max(length(lr_sandwich_by_eth_files) - 3, 0)
)

binary_methods <- list(
  fastGWA_GE_naive = fastgwa_bin_naive,
  LEMMA_naive = lemma_bin_naive,
  GENESIS = genesis_bin,
  GEM = gem_bin,
  MRMEGA_GEM = mrmega_gem
)

binary_summary <- summarize_method_list(
  method_results = binary_methods,
  causal_snps = causal_bin,
  scenario = "toy_binary",
  seed = 100001,
  mrmega_methods = "MRMEGA_GEM",
  mrmega_pc_num = pc_num_bin
)

continuous_summary
#>           method      scenario  seed genomic_inflation_factor_G
#> 1      LRallSandwich toy_continuous 100001          0.9680583
#> 2    meta_LRallSandwich toy_continuous 100001          1.1550683
#> 3 MRMEGA_LRallSandwich toy_continuous 100001          1.0316640
#> 4      fastGWA_GE toy_continuous 100001          0.9861246
#> 5          GENESIS toy_continuous 100001          0.7322637

```

```

#> 6          LEMMA toy_continuous 100001          0.9770279
#> genomic_inflation_factor_G_by_E genomic_inflation_factor_2df      FPR_G
#> 1          0.9469323          1.003596 0.04688547
#> 2          1.1535661          1.363771 0.07970529
#> 3          1.1394478          NA 0.08573342
#> 4          0.9324358          1.019834 0.04755526
#> 5          0.8494997          0.942750 0.01942398
#> 6          0.9696646          NA 0.04487609
#> FPR_G_by_E      FPR_2df power_G power_G_by_E power_2df power_G_pval
#> 1 0.06363027 0.05961152 7.708685 5.559663 10.229024 0
#> 2 0.07568654 0.12659076 5.058862 4.053721 6.603935 0
#> 3 0.08372405          NA 6.006807 5.247079          NA 0
#> 4 0.06697924 0.06229069 7.466928 5.587738 10.047843 0
#> 5 0.05559277 0.05157401 1.602758 6.077606 13.451387 0
#> 6 0.06697924          NA 7.986278 5.568824          NA 0
#> power_G_by_E_pval power_2df_pval
#> 1          0          0
#> 2          0          0
#> 3          0          NA
#> 4          0          0
#> 5          0          0
#> 6          0          NA
binary_summary
#>          method scenario seed genomic_inflation_factor_G
#> 1 fastGWA_GE_naive toy_binary 100001          0.9732635
#> 2          LEMMA_naive toy_binary 100001          0.9562733
#> 3          GENESIS toy_binary 100001          1.1104008
#> 4          GEM toy_binary 100001          1.0617048
#> 5          MRMEGA_GEM toy_binary 100001          1.5861566
#> genomic_inflation_factor_G_by_E genomic_inflation_factor_2df      FPR_G
#> 1          0.9986099          0.9905039 0.04353650
#> 2          1.0483800          NA 0.04085733
#> 3          1.0477505          1.0220596 0.05425318
#> 4          1.0996389          1.0691643 0.05358339
#> 5          1.5551675          NA 0.12458138
#> FPR_G_by_E      FPR_2df power_G power_G_by_E power_2df power_G_pval
#> 1 0.06028131 0.05760214 5.072742 3.432329 8.393848 0
#> 2 0.06430007          NA 4.880383 3.216801          NA 0
#> 3 0.06496986 0.05626256 1.253885 3.196384 7.859967 0
#> 4 0.06430007 0.06095111 4.292676 3.031778 7.171826 0
#> 5 0.12525117          NA 3.484308 3.200806          NA 0
#> power_G_by_E_pval power_2df_pval
#> 1          0          0
#> 2          0          NA
#> 3          0          0
#> 4          0          0
#> 5          0          NA

```